

Oracle Cursors Overview

teach me everything you know about cursors in oracle, how and when to use it, common mistakes with cursor, what exactly is cursor and how it works, what is the purpose of using cursors, how it makes things easy rather than normal plsql

To provide you with the most relevant explanation and examples, could you clarify your current experience level with Oracle PL/SQL? For instance:

1. Are you a beginner, intermediate, or advanced in Oracle PL/SQL?
2. Are you more interested in explicit cursors, implicit cursors, or both?
3. Would you like examples using real-world use cases (e.g., payroll processing, inventory management)?

This will help tailor the depth and focus of the explanation.

somewhat intermediate. i have done plsql before
explicit
yes of course. give me as much detailed example as you can please

Great! I'll put together a comprehensive explanation of explicit cursors in Oracle PL/SQL. This will cover what cursors are, how they work, when and why to use them, common mistakes to avoid, and how they offer advantages over standard SQL in PL/SQL. I'll also include a detailed real-world example such as payroll processing to illustrate best practices.

I'll let you know as soon as it's ready for your review.

 Comprehensive Guide to Explicit Cursors in Oracle PL/SQL



Oracle PL/SQL Explicit Cursors: A Comprehensive Guide

What Are Cursors in PL/SQL?

A *cursor* in Oracle PL/SQL is a pointer to a private SQL area in memory that holds the state of a query or DML operation. The PL/SQL reference defines a cursor as “a pointer to a private SQL area that stores information about processing a specific *SELECT* or *DML* statement.” (A session cursor lives in memory until the session ends .) In practice, every SQL statement executed by PL/SQL uses a cursor behind the scenes. For example, when you execute `SELECT ... INTO` or an `UPDATE` statement, Oracle implicitly opens a cursor to run that SQL. You can also declare your own (explicit) cursors to control query processing.

Under the hood, Oracle allocates memory (a **context area**) for each open cursor to track the parsing, execution, and fetching of rows. The number of open cursors per session is limited by available memory and the initialization parameter `OPEN_CURSORS` . (You can monitor open cursors via the `V$OPEN_CURSOR` view.) When a cursor is opened, Oracle parses the SQL and prepares to fetch rows. As you fetch or loop through the cursor, rows are moved from the SQL area into your PL/SQL program variables. Once done, the cursor must be closed to free resources.

Implicit vs. Explicit Cursors

Oracle distinguishes *implicit* and *explicit* cursors. An **implicit cursor** is automatically created and managed by PL/SQL whenever you execute a SQL statement that returns at most one row or does DML (e.g. `INSERT` , `UPDATE` , `DELETE`). For example, a simple `SELECT col INTO var FROM table` uses an implicit cursor. PL/SQL opens the implicit cursor, fetches the single row (or none), and then closes it. You cannot explicitly control these cursors, but you can query their status with attributes like `SQL%FOUND` , `SQL%NOTFOUND` , and `SQL%ROWCOUNT` . The implicit cursor closes automatically after the statement executes, though its attribute values remain available until the next SQL runs .

By contrast, an **explicit cursor** is defined and controlled by the developer . You declare an explicit cursor with a name and an associated query (typically one that returns multiple rows), for example:

```
plsql
```

```
CURSOR c_emp IS SELECT employee_id, salary FROM employees;
```

Oracle notes that “An explicit cursor is a session cursor that you construct and manage” . After declaration, you must `OPEN` the cursor, `FETCH` rows into variables, and then `CLOSE` it. Unlike implicit cursors, explicit cursors are *named*, so you can refer to them anywhere in your PL/SQL block by name . Explicit cursors typically return multiple rows, enabling you to loop through a result set. In summary:

- **Implicit Cursor** – Created by PL/SQL automatically for single-row queries or DML. You cannot directly open/close it; it closes after execution. Accessible via `SQL%` attributes .
- **Explicit Cursor** – Declared by you with a name and query. You explicitly open it, fetch rows, and close it. Useful for multi-row queries and row-by-row processing .

When and How to Use Explicit Cursors

Use an explicit cursor when you need to process the results of a query one row at a time, especially if the query returns many rows or requires procedural logic per row. Typical use cases include generating reports, calculating payroll/bonuses per employee, updating inventory records in a loop, or any situation where row-by-row processing is required. Explicit cursors give you fine control over row fetching and loop termination. The basic pattern is: declare the cursor with a query, `OPEN` it, then loop fetching each row until no more rows exist, and finally `CLOSE` it .

Here are the general steps using an explicit cursor:

1. **DECLARE** the cursor and any variables to hold columns.
2. **OPEN** the cursor to execute the query.
3. **LOOP** and **FETCH**: In a loop, fetch each row from the cursor into your variables. After each fetch, test the cursor's `%NOTFOUND` (or `%FOUND`) attribute and `EXIT WHEN` no more rows.
4. **PROCESS** the data inside the loop (e.g. calculate values, perform updates).
5. **CLOSE** the cursor when done, to release resources.

This pattern can also be shortened using a *Cursor FOR LOOP*, which implicitly opens the cursor, fetches each row, and closes it when done. Under the hood, a cursor FOR LOOP is equivalent to the explicit OPEN-FETCH-CLOSE pattern, but it makes the code cleaner.

Advantages of Using Explicit Cursors

Explicit cursors offer several advantages over ad-hoc queries or single-row SELECTs. For example, Oracle points out that *“PL/SQL parses an explicit cursor only the first time the session opens it”* . This means if you open the same cursor multiple times in a session (with no changes to the query), Oracle uses a cached parse, saving overhead. Because a cursor has a name, you can reuse the cursor easily without re-entering the SQL text.

Other advantages include:

- **Row-by-Row Control:** You can fetch and process one row at a time, allowing complex logic (conditional processing, calling subroutines, etc.) for each row.
- **Status Attributes:** Cursors expose attributes (`cursor_name%FOUND` , `%NOTFOUND` , `%ROWCOUNT`) to check how many rows have been fetched or whether the last fetch found a row.
- **Flexibility:** Unlike `SELECT ... INTO` (which fails if more than one row is returned), an explicit cursor can handle multiple rows gracefully. It also allows nested loops over different cursors.
- **Readability & Maintainability:** Named cursors make code easier to understand. You can clearly see the query definition and the processing loop.

In short, explicit cursors are invaluable when SQL alone cannot express the needed logic, or when you need to process each row of a multi-row result set.

Example: Processing Employee Bonuses

The following example demonstrates an explicit cursor to calculate and assign a 10% bonus to each employee in a department. It fetches rows one by one, performs a calculation, and updates the table:

```
plsql
```

```
DECLARE
-- Declare an explicit cursor for employees in dept 10
CURSOR emp_cur IS
  SELECT employee_id, salary
```

```

FROM employees
WHERE department_id = 10;
-- Variables to hold cursor output
v_emp_id employees.employee_id%TYPE;
v_salary employees.salary%TYPE;
v_bonus NUMBER(10,2);
BEGIN
OPEN emp_cur;           -- Open the cursor
LOOP
  FETCH emp_cur INTO v_emp_id, v_salary;
  EXIT WHEN emp_cur%NOTFOUND;  -- Exit when no more rows
  -- Calculate bonus and update the table
  v_bonus := v_salary * 0.10;
  UPDATE employees
    SET bonus = v_bonus
    WHERE employee_id = v_emp_id;
END LOOP;
CLOSE emp_cur;          -- Close the cursor
COMMIT;
END;
```

In this example:

- We **DECLARE** `emp_cur` as an explicit cursor for a specific query.
- We **OPEN** `emp_cur` so Oracle executes the query.
- Inside the `LOOP`, each `FETCH` retrieves the next row into `v_emp_id, v_salary`. We immediately test `emp_cur%NOTFOUND` to exit the loop when all rows are processed.
- We **PROCESS** each row (calculating `v_bonus` and issuing an `UPDATE` for that employee).
- After the loop, we **CLOSE** the cursor to free resources.

This follows the documented pattern: open, fetch, and close the explicit cursor. It allows us to handle as many rows as the query returns, one at a time, and apply custom logic to each.

Common Mistakes with Explicit Cursors

- **Forgetting to Close the Cursor:** If you `OPEN` a cursor and don't `CLOSE` it, you can exhaust the session's cursor limit (`OPEN_CURSORS`) and cause memory leaks. Always ensure every code path closes the cursor (use `EXIT` carefully or wrap in `BEGIN...EXCEPTION...END` with a close in the `EXCEPTION` or `FINALLY` section).
- **Incorrect Loop Exit Condition:** A common error is placing `EXIT WHEN %NOTFOUND` incorrectly. You must test `%NOTFOUND` *after* each `FETCH` . Exiting before the first fetch or checking the wrong variable can cause infinite loops or skipped rows.
- **Mismatched FETCH Into Variables:** Ensure the `INTO` list matches the `SELECT` list (same number of columns and compatible types). Mismatches cause runtime errors (e.g. "ORA-01002: fetch out of sequence" or "ORA-00947: not enough values").
- **Not Opening the Cursor:** Attempting to `FETCH` without `OPEN` ing the cursor raises ORA-01001. Always open before fetching.
- **Unnecessary Cursors:** Using an explicit cursor for a single-row query or when a simple `SELECT ... INTO` would suffice is overkill. Conversely, using `SELECT ... INTO` when multiple rows are expected causes exceptions. Choose the correct approach.
- **Locking Issues:** Updating rows inside a cursor loop can lead to locking conflicts. If possible, minimize DML inside loops, or collect IDs then do a bulk update outside the loop.
- **No Bulk Processing:** For very large datasets, row-by-row `FETCH` and `UPDATE` is slow. Consider bulk features (see below) or set-based SQL if appropriate.

Performance Considerations and Optimization

Explicit cursors inherently involve context switches between PL/SQL and SQL for each fetch, which can be a performance bottleneck. However, modern Oracle optimizations help mitigate this. For example, as noted by expert Pavel Smirnov, starting in Oracle 10g the PL/SQL compiler can automatically convert a cursor `FOR` loop into a bulk array fetch of 100 rows at a time . This means simple loops are often optimized behind the scenes.

Nonetheless, for high-volume row processing, consider these tips:

- **Use BULK COLLECT:** Instead of fetching one row at a time, fetch into a PL/SQL collection in chunks. For example:

```
plsql
```

```

OPEN emp_cur;
LOOP
  FETCH emp_cur BULK COLLECT INTO l_emp_ids, l_salaries LIMIT 100;
  EXIT WHEN emp_cur%NOTFOUND;
  -- process collection of rows here
END LOOP;
CLOSE emp_cur;

```

Bulk collect reduces the number of context switches between PL/SQL and SQL, greatly speeding up large queries.

- **Use FORALL for DML:** If you need to perform DML (INSERT/UPDATE/DELETE) for many fetched rows, combine `BULK COLLECT` with `FORALL`. This applies DML to all rows in a collection in one statement, which is much faster than row-by-row updates.
- **Tune Array Fetch Size:** The `LIMIT` in `BULK COLLECT` controls how many rows are fetched per round-trip. The optimal size depends on data volume and memory, but common sizes are 100–1000.
- **Leverage Set-Based SQL:** Whenever possible, do work with a single SQL statement instead of a cursor loop. For instance, an `UPDATE` that joins tables or a `MERGE` statement can often replace a loop. As one authority notes, *“Whenever possible you should prefer to process your data with single SQL statements (consider also MERGE... not only DELETE, UPDATE, INSERT), in most cases they are faster than row-by-row processing.”* . Set-based operations run inside the database engine and avoid PL/SQL overhead entirely.
- **Monitor OPEN_CURSORS:** If you open many cursors (especially in PL/SQL packages or loops), ensure the `OPEN_CURSORS` parameter is sufficiently high, or close cursors promptly to avoid errors.

In summary, explicit cursors are powerful for control, but for pure performance on large data sets, bulk operations and set-based SQL are often more efficient . Use explicit cursors judiciously when row-by-row logic is truly needed.